

What is Python?

Python is a high-level, general-purpose programming language known for its clear, English-like syntax and ease of readability

Features: Simple syntax | Interpreted execution | Cross-platform compatibility | Open source library access

Uses: Artificial intelligence | Web development backend | Data science and analysis | Repetitive task automation

Variables

In Python, a variable is just like a container to store the data and used to store and reference data in a computer's memory. Instead of remembering a complex memory address, you give that location a descriptive name so you can easily reuse and change the data throughout your code.

Example:

```
name = "rohit"  
age = 21
```

A variable name in Python must start with a letter or underscore (_) and can contain letters, numbers, and underscores. It cannot start with a number, contain spaces, or use Python keywords such as `if`, `for`, or `class`. Variable names are case-sensitive, so `name` and `Name` are different.

Data types

Data types define what kind of value a variable contains.

```
name = "Rohit"      # str  
age = 21             # int  
marks = 85.5         # float  
is_active = True     # bool
```

In Python, data types can be classified based on whether their value can be changed after creation.

Immutable

Immutable means the existing object cannot be changed after it is created.

Common immutable types:

Data Type	Example
Integer	10
Float	10.5
Boolean	True
String	"Rohit"
Tuple	(1, 2, 3)
Frozenset	frozenset({1, 2, 3})

Mutable

Mutable means the existing object can be changed after it is created.

Common mutable types:

Data Type	Example
List	[1, 2, 3]
Dictionary	{"name": "Rohit"}
Set	{1, 2, 3}

Strings

A string is a sequence of characters used to store text.

Strings are written inside quotes it can be double or single quotes:

```
name = "Rohit"  
course = 'Python'
```

String Slicing

String slicing means taking a specific portion of a string.

The basic syntax is:

```
string[start:stop:step]
```

Important: stop is not included.

slicing

```
name = "Rohit"  
  
print(name[0:3])
```

Output:

Roh

Indexes:

R	o	h	i	t
0	1	2	3	4

name[0:3] means:

```
start = 0  
stop  = 3
```

So Python takes indexes:

0, 1, 2

but not 3.

Start from a specific index

```
name = "Rohit"  
print(name[2:])
```

Output:

hit

Meaning:

2 → start here
nothing after : → go until the end

Stop at a specific index

```
name = "Rohit"  
print(name[:3])
```

Output:

Roh

Meaning:

nothing before : → start from beginning
3 → stop before index 3

Numbers

Numbers in Python are used to store numeric values and perform calculations.

Python mainly has 3 numeric data types:

1. Integer (int)

Integers are whole numbers without decimal points.

```
age = 21  
student_id = 101  
marks = 85
```

You can check the type:

```
print(type(age))
```

2. Float (float)

A float represents a number with a decimal point.

```
percentage = 85.5  
price = 99.99  
temperature = -2.5
```

Example:

```
x = 10.5  
print(type(x))
```

Output:

```
<class 'float'>
```

3. Complex (complex)

A complex number has a real part and an imaginary part.

```
x = 3 + 4j
```

Here:

```
3 → real part  
4j → imaginary part
```

Lists

A list is a collection used to store multiple values in a single variable.

Lists are:

- Ordered
- Mutable
- Allow duplicate values
- Can contain different data types
- Indexed starting from 0

```
example l = ["rohit" , 4 , "hello"]
```

Useful List Methods

Method	Use
<code>append()</code>	Add at end
<code>insert()</code>	Add at position
<code>remove()</code>	Remove by value
<code>pop()</code>	Remove by index
<code>clear()</code>	Remove everything
<code>sort()</code>	Sort the list
<code>reverse()</code>	Reverse the list
<code>index()</code>	Find position
<code>count()</code>	Count occurrences

Modifying a List

Lists are mutable, so their contents can be changed.

Change an element

```
s[1] = "new value"

print(s)
```

List Length

Use `len()`:

```
print(len(students))
```

It tells you how many elements are present.

Checking an Element

Use the `in` operator:

```
if "Rohit" in students:
    print("Student found")
```

Looping Through a List

Use a `for` loop:

```
for student in students:
    print(student)
```

List of Dictionaries

we can store multiple students like this:

```
students = [
    {
        "id": 1,
        "name": "Rohit",
        "age": 21
    },
    {
        "id": 2,
        "name": "Rahul",
        "age": 20
    }
]
```

List Slicing

Lists use the same slicing syntax as strings.

```
list[start:stop:step]
```

The `stop` index is not included.

Tuples

A tuple is an ordered collection of values that is immutable.

Creating Tuple

```
student = ("Rohit", 21, "CSE")
```

Characteristics

- Ordered
- Indexed
- Immutable
- Allows duplicates
- Can store different data types
- Supports slicing
- Written using `()`

Indexing

```
student[0]      # Rohit
student[-1]     # CSE
```

Index starts from 0.

Slicing

```
student[0:2]    # ("Rohit", 21)
student[:2]     # ("Rohit", 21)
student[1:]     # (21, "CSE")
student[::-1]   # Reverse tuple
```

Duplicates

```
numbers = (10, 20, 10, 30)
```

Duplicates are allowed.

Immutable

We cannot change an existing element:

```
student[0] = "Rahul"    Error
```

Common Methods

```
numbers.count(10)    # Count value
numbers.index(20)     # Find index
```

Tuples have fewer methods because they are immutable.

Packing & Unpacking

```
student = ("Rohit", 21, "CSE")
```

```
name, age, course = student
```

Single-Element Tuple

A comma is required:

```
x = (10,)      # Tuple  
x = (10)      # Integer
```

Sets

A set is a collection used to store unique values.

Characteristics

- Unordered
- Not indexed
- Mutable
- Does not allow duplicates
- Can contain different data types
- Supports adding and removing elements
- Written using {}

Creating a Set

```
skills = {"Python", "SQL", "Git"}
```

Duplicates are automatically removed:

```
skills = {"Python", "SQL", "Python"}
```

```
print(skills)
```

No Indexing

You cannot do:

```
skills[0]    # Error
```

Because sets are not indexed.

You can use a loop:

```
for skill in skills:  
    print(skill)
```

Adding Elements

```
skills.add("AWS")
```

Removing Elements

```
skills.remove("SQL")
```

If the value doesn't exist, `remove()` raises an error.

```
skills.discard("SQL")
```

`discard()` does not raise an error if the value doesn't exist.

METHODS

<code>add()</code>	→ Add one
<code>update()</code>	→ Add multiple
<code>remove()</code>	→ Remove
<code>discard()</code>	→ Remove safely
<code>pop()</code> not specifically chosen)	→ Remove arbitrary element(not fixed or
<code>clear()</code>	→ Remove all
<code>union()</code>	→ Combine
<code>intersection()</code>	→ Common elements
<code>difference()</code>	→ Difference
<code>symmetric_difference()</code>	→ Non-common elements
<code>issubset()</code>	→ Check subset
<code>issuperset()</code>	→ Check superset
<code>isdisjoint()</code>	→ Check no common elements
<code>copy()</code>	→ Copy set

Empty Set

Notes:

```
x = {}      # Dictionary  
x = set()   # Empty set
```

Dictionaries

A dictionary is a collection that stores data in key-value pairs.

```
student = {  
    "id": 101,  
    "name": "Rohit",  
    "age": 21  
}
```

Characteristics

- Mutable
- Ordered (insertion order is preserved)
- Accessed using keys, not indexes
- Keys must be unique
- Values can be duplicated
- Values can be any data type
- Written using {}

Access Values

```
print(student["name"])
```

Output:

Rohit

You can also use `get()`:

```
print(student.get("name"))
```

`get()` is safer when the key may not exist it gives none.

Add / Update

Add a new key:

```
student["course"] = "CSE"
```

Update an existing key:

```
student["age"] = 22
```

Delete

```
student.pop("age")
```

Removes the specified key and returns its value.

```
del student["course"]
```

Deletes the key-value pair.

```
student.clear()
```

Removes everything.

Check Key

```
if "name" in student:  
    print("Name exists")
```

Loop Through Dictionary

Keys:

```
for key in student:  
    print(key)
```

Values:

```
for value in student.values():  
    print(value)
```

Both:

```
for key, value in student.items():  
    print(key, value)
```

Important Dictionary Methods

keys()	→ Get all keys
values()	→ Get all values
items()	→ Get key-value pairs
get()	→ Get value safely
update()	→ Add/update multiple values
pop()	→ Remove specified key
popitem()	→ Remove last inserted pair
clear()	→ Remove all
copy()	→ Create a copy

Nested Dictionary

A dictionary can contain another dictionary:

```
students = {  
    1: {  
        "name": "Rohit",  
        "age": 21  
    },  
    2: {  
        "name": "Rahul",  
        "age": 20  
    }  
}
```

Conditions

Conditions are used to make decisions in a Python program.

if

Runs code when a condition is True.

```
age = 21
```

```
if age >= 18:
```

```
print("Adult")
```

if-else

If the condition is true, **if** runs; otherwise, **else** runs.

```
age = 16

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

if-elif-else

Used when there are multiple conditions.

```
marks = 75

if marks >= 90:
    print("A")
elif marks >= 60:
    print("B")
else:
    print("C")
```

match-case is Python's way of doing pattern matching. It is similar to **switch-case** in languages like C, C++, and Java.

Example:

```
c = 2

match c:
    case 1:
        print("Add Student")
    case 2:
        print("View Students")
    case 3:
        print("Search Student")
    case _:
        print("Invalid choice")
```

Output:

View Students

Loops

Loops are used to **repeat a block of code**.

for loop

Used to iterate over a sequence or repeat a known number of times.

```
for student in students:
    print(student)
```

```
for i in range(5):  
    print(i)
```

while loop

Runs while a condition is True.

```
count = 1  
  
while count <= 5:  
    print(count)  
    count += 1
```

break

Stops the loop completely.

```
for i in range(10):  
    if i == 5:  
        break
```

continue

Skips the current iteration.

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

range()

Used to generate a sequence of numbers.

```
range(start, stop, step)
```

Example:

```
range(1, 6)  
# 1, 2, 3, 4, 5
```

Nested Loop

A loop inside another loop.

```
for i in range(2):  
    for j in range(3):  
        print(i, j)
```